

Justificativa da escolha de banco de dados — Conexão Solidária

Hackathon Fase 5 — POSTECH .NET, Turma 11NETT, Grupo 30

Versão resumida, focada na escolha de banco de dados em si. Para o raciocínio completo (incluindo o papel da fila de mensageria e a garantia de idempotência), ver [justificativa-banco-dados-completa.pdf](#).

Por que um banco relacional (PostgreSQL)

Doações e campanhas são, na essência, registros financeiros: uma doação tem um valor e um status que precisa refletir com exatidão se ela foi efetivamente confirmada, e o valor arrecadado de uma campanha é a soma confiável dessas doações. Esse tipo de dado exige as garantias clássicas de um banco relacional com suporte a transações ACID — não há espaço para "quase certeza": o valor arrecadado de uma campanha precisa corresponder exatamente à soma das doações confirmadas. Um banco não relacional, otimizado para escala e disponibilidade em detrimento de consistência forte, resolveria um problema que o Conexão Solidária não tem, e abriria mão exatamente da garantia mais importante para o domínio. Por isso o grupo optou pelo PostgreSQL, um banco relacional maduro, com suporte transacional completo e amplamente compatível com o ecossistema .NET/EF Core usado no projeto.

Por que uma única instância com três schemas, em vez de três bancos separados

O sistema é organizado em três áreas de responsabilidade — Identidade, Campanha e Doação —, e cada uma poderia, em tese, ter seu próprio banco fisicamente isolado. O grupo decidiu não seguir esse caminho: o projeto usa **uma única instância do PostgreSQL, dividida em três schemas** (identidade, campanha e doacao), preservando a separação lógica entre os módulos sem pagar o preço de dados fisicamente espalhados.

O motivo central está no momento em que uma doação é processada: o valor arrecadado da campanha precisa ser incrementado e a doação precisa ser marcada como confirmada — os dois juntos, sem exceção. Se esses dados morassem em bancos fisicamente diferentes, garantir isso exigiria coordenar duas transações independentes (transações distribuídas, outbox pattern ou saga), técnicas que resolvem um problema real mas adicionam uma camada de complexidade de implementação, teste e operação incompatível com o prazo do hackathon. Com os schemas na mesma instância, o Worker responsável por processar a doação abre **uma única transação real do banco**, que atualiza a campanha e confirma a doação ao mesmo tempo: ou as duas mudanças são gravadas com sucesso, ou nenhuma delas é — sem risco de o sistema ficar num estado intermediário.

Conclusão

O PostgreSQL foi escolhido pelas garantias ACID que o domínio financeiro do projeto exige; a organização em um único banco com schema por módulo (em vez de bancos separados) foi escolhida para permitir uma transação atômica real cobrindo Campanha e Doação, sem precisar reconstruir manualmente, em nível de aplicação, uma garantia que o próprio PostgreSQL já resolve de forma nativa.